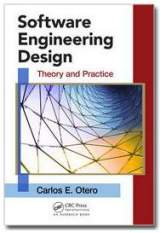
Chapter 7: Structural and Behavioral Design Patterns

Session I: Adapter, Composite, Facade

Software Engineering Design: Theory and Practice
by Carlos E. Otero



Slides copyright © 2012 by Carlos E. Otero

For non-profit educational use only

May be reproduced only for student use when used in conjunction with *Software Engineering Design: Theory and Practice.* Any other reproduction or use is prohibited without the express written permission of the author.

All copyright information must appear if these slides are posted on a website for student use.

SESSION'S AGENDA

- Structural Patterns in Detailed Design
- Adapter
  - ✓ Character adapter example
- Composite
  - ✓ Message generator example
- Facade
  - ✓ Subsystem interface example
- What's next…

ADAPTER DESIGN PATTERN

- The step-by-step approach for designing the object structural adapter design pattern is presented as follows:
  1. Identify the source and destination interfaces that need adapting in the new system (e.g., Target and Adaptee or Character and SpecialCharacter).
  2. Add a new class (e.g., Adapter or AdaptedCharacter) in the design that realizes the Target interface and implements it in terms of the Adaptee's implementation. This requires a realization relationship between the Adapter and Target, and an association between the Adapter and the Adaptee.
  3. In the new system, whenever objects that share the Target interface are expected, it can now be possible to use the Adapter objects created in step 2.

- Benefits of the Adapter design pattern include:
  - ✓ Allows classes with incompatible interfaces to work together, therefore it increases reusability and ease of code integration.
  - ✓ Provides a standard way for integrating a plurality of different types to existing software.

Adapter Design Pattern

•**Description**:
The Adapter Pattern acts as a bridge between two incompatible interfaces by converting one interface into another that the client expects.
•**When to Use**:
- When you want to integrate classes with incompatible interfaces.
- To reuse existing code that doesn't work well with other parts of the system.

•**Why Use**:
To promote code reuse and ensure compatibility between mismatched systems.
•**Scenarios**:
- Integrating legacy systems with new systems.
- Using third-party APIs in your application.

•**Pros**:
- Simplifies integration between incompatible interfaces.
- Promotes reusability.

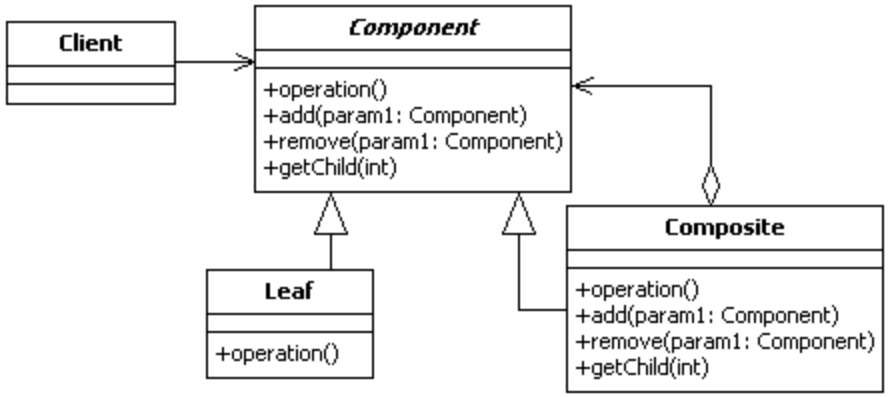
•**Cons**:
- Can add complexity.
- May introduce extra layers of abstraction.

Adapter Design Pattern

COMPOSITE DESIGN PATTERN

- The Composite design pattern is an object structural pattern that allows designers to compose (large) tree-like designs structures by strategically structuring objects that share a whole-part relationship.

- According to the GoF, the intent of the Composite is to [1]
  - ✓ Compose objects into tree structures to represent whole-part hierarchies. Composite lets clients treat individual objects and composites of objects uniformly.



Composite Design Pattern

- The steps required to apply the composite design pattern include:
  1. Identify, understand, and plan the tree-like structure required for the system.
  2. With the knowledge from step 1, identify and design the Component base class (see diagram), which includes overridable methods common to both Leaf and Composite objects, as well as methods specific for Composite objects, which provide capability for adding and removing objects to the hierarchy.
  3. For the methods specified in step 2 for adding/removing objects to the hierarchy, implement default behavior that if not overridden, will result in exception or error message indicating an unsupported operation.
  4. Identify and design the Composite class, which overrides methods for adding and removing objects to the hierarchy. The Composite class requires an internal data structure to store Leaf nodes added to the hierarchy. In addition, the Composite class is required to override all other methods identified in step 2 to implement functionality in terms of the composite object and all of its contained Leaf objects.
  5. Identify and design the Leaf classes, which overrides methods specified in step 2 to implement behavior in terms of the Leaf object. Leaf objects do not override the add and remove methods identified in step 2.
  6. Identify and design the client that uses both composite and leaf objects.

- Benefits of the Composite design pattern:
  - ✓ Provides a design structure that supports both composite and primitive objects.
  - ✓ Minimizes complexity on clients by shielding them from knowing the operational differences between primitive and composite objects. Clients that expect a primitive object will also work with a composite object, since operations are called uniformly on both primitive and composite objects.
  - ✓ Easy to create and add new components objects to applications.

Composite Design Pattern

•**Description**:
The Composite Pattern composes objects into tree-like structures to represent part-whole hierarchies. It allows clients to treat individual objects and compositions uniformly.

•**When to Use**:

- When you need to represent hierarchies like file systems, organizational structures, or UIs.
- When clients need to treat individual and composite objects uniformly.

•**Why Use**:
To make it easier to add new types of components and process them uniformly.

•**Scenarios**:

- Creating a GUI with nested elements (e.g., windows, panels, buttons).
- Managing a hierarchy of organizational roles.

•**Pros**:

- Simplifies client code by treating composites and leaf objects uniformly.
- Makes adding new component types easy.

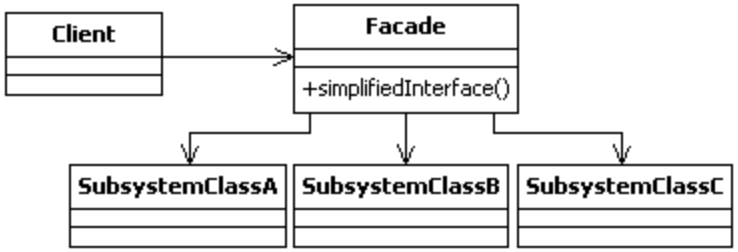
•**Cons**:

- Can make the system overly general.
- Can be challenging to enforce constraints on the composition.

Facade Design Pattern

- The Facade design pattern is an object structural pattern that provides a simplified interface to complex subsystems.

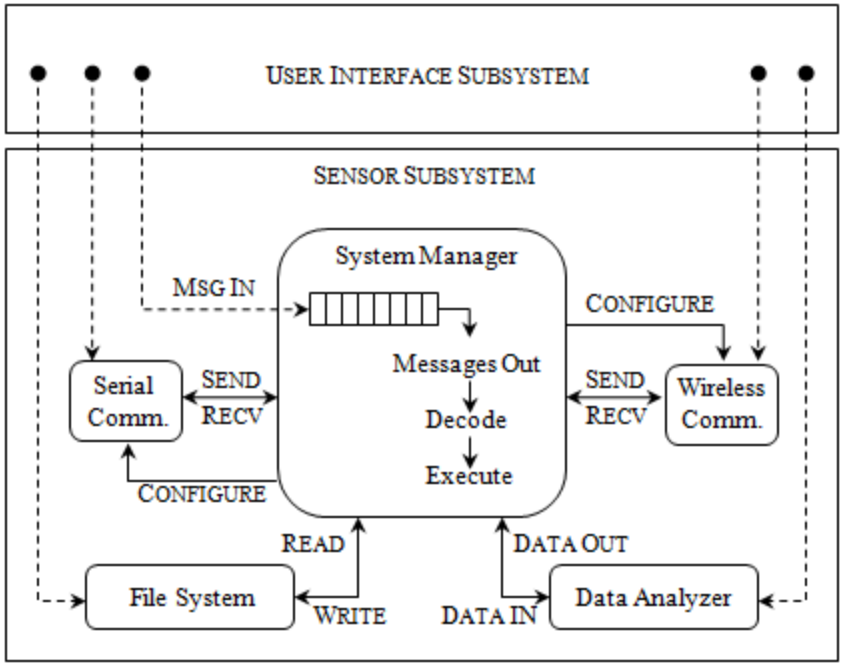
- According to the GoF, the intent of the Facade is to [1]
  - ✓ Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use.



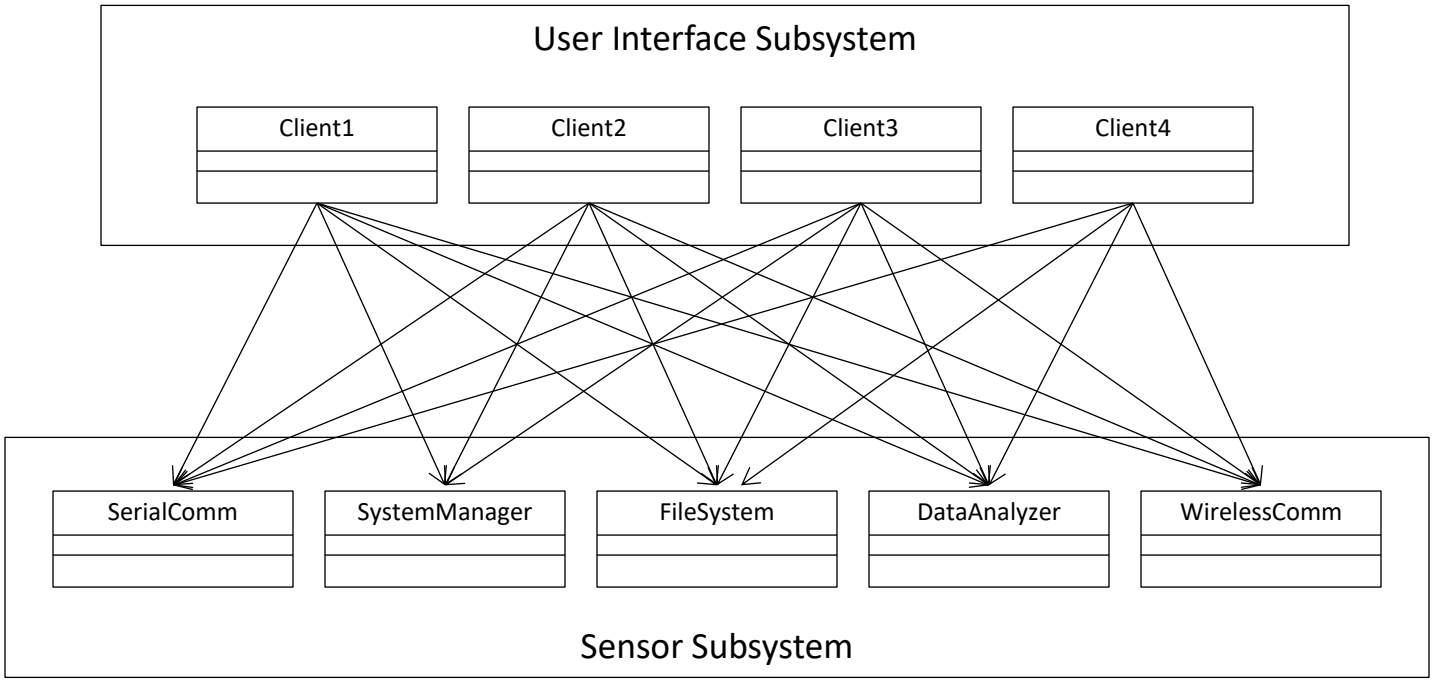
FACADE DESIGN PATTERN EXAMPLE

Problem

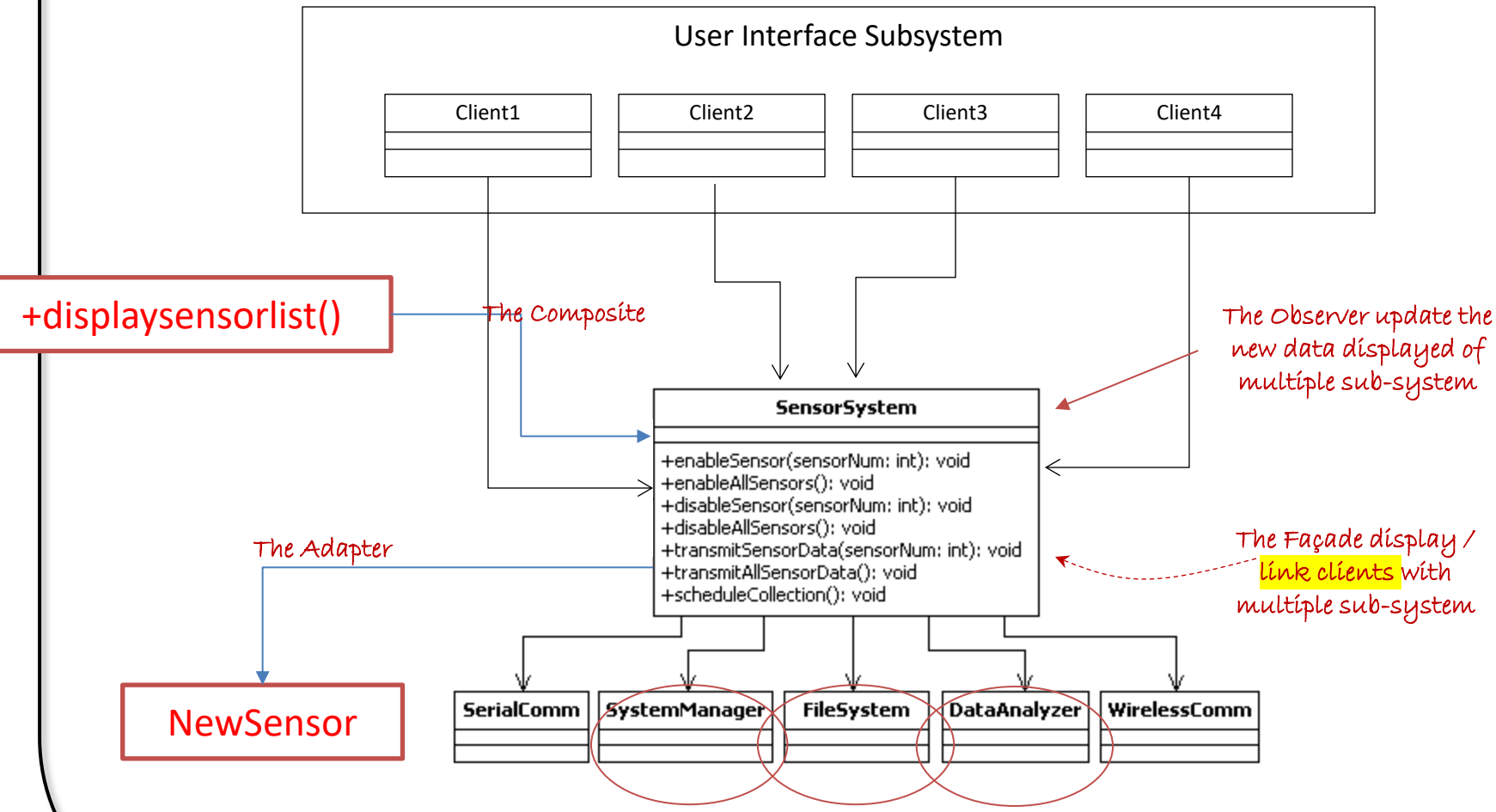
Consider the sensor system described as part of the message generator in the previous section. Upon field deployment, it is desirable to test the system's capabilities to ensure that the system works properly before engaging in autonomous operation. For this reason, a graphical user interface is required to monitor and control the system in the field during installation.



FACADE DESIGN PATTERN EXAMPLE



Facade Design Pattern Example



Facade Design Pattern Example

The Façade hides all of these details and dependencies from clients, therefore simplifying the client side.

```
void SensorSystem::transmitSensorData(int sensorNumber) {

  // Create an object for serial communications parameters.
  SerialParams params;
  params.setCommPort( SerialParams::COM_1 );
  params.setBaudRate( SerialParams::BR_9600 );
  params.setParity( SerialParams::PARITY_NO_PARITY );
  params.setByteSize( SerialParams::BYTE_SIZE_8 );
  params.setStopBits( SerialParams::STOP_BIT_ONE );

  // Retrieve pointer to the serial communication object.
  SerialComm* pSerialComm = SerialComm::getInstance();

  // Open the serial communication with the specified parameters.
  if( serialComm->open(params) ) {

    // Ready to communicate with collection nodes, now get ready for
    // transmitting the data via the wireless link.
    TcpConnection* pConnection = TcpConnection::getInstance();

    if( pConnection->open(TcpConnection::PORT_NUMBER,
                          TcpConnection::IP_ADDRESS) ) {

      // Schedule a collection message.
      SystemManager::getInstance()->scheduleMessage(/*...*/);
    }
    else {
      // Log TCP error here.
      // Close serial connection.

    } // end if( pConnection->open(...)
  }
  else {
    // Log serial connection error here.
  } // end if( serialComm->open(...)
} // end transmitSensorData function.
```

FACADE DESIGN PATTERN

- The step-by-step procedure for applying the Facade design pattern include:
  1. Identify all components involved in carrying out a subsystem operation
  2. Create an ordered list of the operations required to execute the subsystem operation.
  3. Design a Facade class that includes an interface method to carry out the subsystem operation. The Facade class has dependencies to all other subsystem components required to carry out the subsystem operation.
  4. Implement the Facade interface method by calling operations on one or more subsystem components, in the order identified in step 2.
  5. Allow one ore more clients to access the objects of the Facade type so that they can gain access to the subsystem operation. This creates a many-to-one relationship between external subsystems and the Facade interface, instead of many-to-many relationships.

- Benefits of the Facade design pattern include:
  - ✓ Shields clients from knowing the internals of complex subsystem, therefore minimizing complexity in clients.
  - ✓ Since the internals of the subsystem are prone to change, the facade provides a stable interface that hides changes to internal subsystems; therefore, client code is more stable.
  - ✓ Promotes weak coupling on clients; with facade, clients depend only on one interface instead of multiple interfaces.

Facade Design Pattern

•**Description**:
The Facade Pattern provides a simplified interface to a complex subsystem by hiding its complexity.

•**When to Use**:

- When a system is complex, and you need to provide a simpler interface for clients.
- To decouple subsystems from the client code.

•**Why Use**:
To reduce the learning curve and dependency for clients interacting with complex subsystems.

•**Scenarios**:

- Simplifying database access layers.
- Providing a unified API for multiple libraries or services.

•**Pros**:

- Reduces complexity for clients.
- Decouples subsystems from higher-level code.

•**Cons**:

- May become a "god object" if not carefully designed.
- Adds an additional layer of abstraction.

SUMMARY

- Creational pattern more to adding new object with repetitive procedure steps/methods.
- Structural pattern is more like integration action by user like adding new component with different procedure/method.
- Behavioral pattern, more to system behavior toward data, normally not directly trigger by user action.

WHAT'S NEXT…

- In this session, we presented structural design patterns, including:
  - ✓ Adapter
  - ✓ Composite
  - ✓ Facade

- The next session continues the discussion on patterns by presenting a different category of design patterns capable of encapsulating behavior. These behavioral patterns include:
  - ✓ Iterator
  - ✓ Observer

REFERENCES

- [1] Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley, 1995.